



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

**SMART ROUTE OPTIMIZATION USING DIJKSTRA'S
ALGORITHM FOR INTELLIGENT NAVIGATION SYSTEMS**

A CAPSTONE PROJECT REPORT

*A Capstone Project Report submitted in partial fulfilment of the requirements for
the Course of*

CSA0622 – DESIGN AND ANALYSIS OF ALGORITHMS

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted by

P. Muni Karthik – 192425061

Under the Supervision of

Dr. D. Manikavelan

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER -2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “ **Smart Route Optimization Using Dijkstra’s Algorithm For Intelligent Navigation Systems** ” has been carried out by P. MUNI KARTHIK (192425061), CHAMAKURI SRIRAM(192372279), P. KESAVA REDDY (192525088 under the supervision of Dr. D. MANIKVELAN and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech – AIML Engineering program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr. F.Mary Harin Fernandez

Professor

Department of CSE,

Saveetha School of Engineering

SIMATS, Chennai

COURSE FACULTY

Dr. MANIKVELAN

Professor

Department of CSE,

Saveetha School of Engineering,

SIMATS, Chennai

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences Chennai-602105

DECLARATION

I, P. MUNI KARTHIK of the AIML, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “**Smart Route Optimization Using Dijkstra’s Algorithm For Intelligent Navigation Systems**” is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date: _____

Signature of Student:

P. Muni Karthik

192425061

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences Chennai-602105

Individual Contribution Statement

The team project was divided into three major parts. The assigned responsibilities were distributed among algorithm development, graph representation and route processing, and testing, analysis and documentation. Each team member contributed to the design, implementation, testing and documentation of the intelligent navigation system.

Student	Specific Responsibilities	Design & Development	Testing & Analysis	Report Contribution	Approx . (%)
Chamakuri Sriram 192372279	Part 1 – Navigation Network & Graph Modeling	Design of graph structure; creation of vertices and weighted edges; adjacency-list representation; input handling	Testing graph construction, node and edge validation	Chapters 1–5, graph modelling, testing and individual reflection	33.3%
P. Muni Karthik 192425061	Part 2 – Dijkstra Algorithm & Route Optimization	Implementation of Dijkstra's algorithm; priority queue; distance calculation; edge relaxation	Verification of shortest-path distances and route correctness	Chapters 1–5, algorithm implementation, analysis and individual reflection	33.3%
P. Kesava Reddy 192525088	Part 3 – Route Reconstruction, Performance Analysis & Integration	Path reconstruction; route display; performance analysis; system integration	Testing route reconstruction, execution performance and final outputs	Chapters 1–5, performance analysis, testing and individual reflection	33.3%
TOTAL					100%

ABSTRACT

Navigation systems need efficient methods to determine the best route between two locations. When a network contains multiple possible paths, identifying the route with the minimum total distance or cost becomes a shortest-path problem. The Smart Route Optimization system addresses this problem by applying Dijkstra's Algorithm to a weighted navigation graph.

My individual contribution focuses on the **Dijkstra Algorithm & Route Optimization** component of the system. The algorithm receives the weighted graph and calculates the minimum distance from the selected source to the reachable vertices. The source vertex is assigned a distance of zero, while the remaining vertices are initially assigned an appropriately large distance value. A priority queue is used to select the vertex having the smallest tentative distance for processing.

The main processing operation is edge relaxation. For each selected vertex, the algorithm examines its connected edges and checks whether travelling through the current vertex produces a shorter distance to a neighbouring vertex. If a shorter distance is identified, the corresponding distance value is updated and the vertex is inserted into the priority queue for further processing.

The implementation is designed for navigation networks where road distances or costs are non-negative. The calculated distances are verified using test cases containing different source-destination combinations and weighted routes. The results demonstrate that Dijkstra's Algorithm can systematically identify the minimum-cost path without requiring manual comparison of all possible routes.

The project demonstrates the application of a fundamental Design and Analysis of Algorithms technique to an intelligent navigation problem. The individual implementation strengthens the connection between greedy algorithm design, graph processing, priority queues, shortest-path computation and practical route optimization.

Keywords: Dijkstra's Algorithm; Route Optimization; Shortest Path; Weighted Graph; Priority Queue; Edge Relaxation; Navigation System; Greedy Algorithm

TABLE OF CONTENTS

Sl. No	Title	Page No
i	CERTIFICATE FROM THE SUPERVISOR	ii
ii	DECLARATION BY THE CANDIDATE	iii
iii	INDIVIDUAL CONTRIBUTION STATEMENT	iv
iv	ABSTRACT	v
v	LIST OF FIGURES	vi
vi	LIST OF TABLES	vii
vii	LIST OF ABBREVIATIONS	viii
1	CHAPTER 1: Introduction	1
2	CHAPTER 2: Literature Review	4
3	CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	7
4	CHAPTER 4: Implementation, Testing & Results, Project Management	12
5	CHAPTER 5: Conclusion & Reflection	18
	References	19
	Appendices	20

LIST OF TABLES

Table No.	Table Name
Table 1	Individual Contribution Statement
Table 2	Project Objectives
Table 3	Stakeholder and User Requirements
Table 4	Functional and Non-Functional Requirements
Table 5	Design Specifications
Table 6	Alternative Solution Evaluation
Table 7	Trade-off Analysis
Table 8	Sustainability, Ethics, Safety and Security Considerations
Table 9	Risk Register
Table 10	Software Tools and Resources
Table 11	Test Plan and Verification
Table 12	Objective Achievement
Table 13	Project Milestones
Table 14	Team Responsibilities
Table 15	Project Budget

LIST OF FIGURES

Figure No.	Figure Name
Figure 1	Dijkstra algorithm and route-optimization module flow
Figure 2	Weighted graph construction
Figure 3	Edge relaxation and distance update process
Figure 4	Shortest-path calculation output
Figure 5	Route optimization result

LIST OF ABBREVIATIONS / SYMBOLS

Symbol / Term	Description
DAA	Design and Analysis of Algorithms
C	Programming language used for the implementation
UI	User Interface
UX	User Experience
I/O	Input / Output
PQ	Priority Queue
V	Number of vertices in the graph
E	Number of edges in the graph
∞	Initial representation of an unknown distance
Dijkstra	Shortest-path algorithm for weighted graphs with non-negative edge weights
Relaxation	Process of checking and updating a shorter known distance
Min-Heap	Heap-based structure used to implement the priority queue

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Navigation systems are used to determine efficient routes between different locations. A navigation network can be represented as a weighted graph in which locations are represented as vertices and roads are represented as edges. Each edge is associated with a weight representing a distance, travel cost or other non-negative route value.

Dijkstra's Algorithm provides a systematic method for solving this problem when edge weights are non-negative. It follows a greedy approach in which the vertex with the smallest currently known distance is selected for processing. The algorithm then examines neighbouring vertices and performs edge relaxation to determine whether shorter paths can be obtained.

My individual contribution focuses on the implementation of this shortest-path calculation. The Dijkstra module forms the algorithmic core responsible for calculating minimum distances from the selected source and supporting the route-optimization process.

1.2 Need for the Project

The project is needed to provide an efficient algorithmic solution for determining minimum-cost routes in a weighted navigation network. When several roads connect different locations, selecting an appropriate route requires calculating the cumulative weight of the available paths.

A manual approach becomes increasingly inefficient as the number of locations and roads increases. An algorithmic solution can process the network systematically and determine the minimum distance from the source.

The Dijkstra module is particularly important because it converts the weighted graph into minimum-distance information that can be used for route optimization. The use of a priority queue also provides an efficient mechanism for selecting the next vertex with the smallest tentative distance.

1.3 Problem Statement

The engineering problem is to develop and validate a shortest-path processing component that receives a weighted navigation graph and determines the minimum distance from a specified source to the required destination.

A navigation network can contain multiple possible paths between two locations. Each road has an associated non-negative weight, such as distance or travel cost. Without an efficient shortest-path algorithm, determining the minimum route may require repeated comparison of different possible paths.

The Dijkstra module must initialize distance values, select the vertex with the smallest tentative distance, examine its neighbouring edges, perform edge relaxation and update shorter distances until the required shortest-path information is obtained.

Design and implement the Dijkstra Algorithm and route-optimization component for a weighted navigation system to calculate minimum distances from a selected source, efficiently process vertices using a priority queue, perform edge relaxation and provide verified shortest-path results for navigation.

1.4 Aim

To develop and validate a Dijkstra-based route-optimization component that efficiently calculates minimum distances in a weighted navigation graph and determines the shortest route cost between a specified source and destination. The objective is to implement the algorithm using a systematic shortest-path approach in which vertices are processed based on their current minimum tentative distance.

The component focuses on initializing distance values, selecting the minimum-distance vertex, processing neighbouring vertices, performing edge relaxation and updating distances whenever a shorter route is identified. The implementation is intended to provide accurate and verifiable route costs for different source-destination combinations within the weighted navigation network.

1.5 Objectives

Objective	Measurable Outcome
Dijkstra Implementation	Implement Dijkstra's Algorithm for a weighted graph containing non-negative edge weights.
Distance Calculation	Calculate the minimum distance from the selected source to reachable vertices.
Priority Queue Processing	Use a priority queue to efficiently select the vertex with the smallest tentative distance.
Edge Relaxation	Update neighbouring vertex distances whenever a shorter route is identified.
Shortest-Path Verification	Compare calculated distances against manually determined expected results.
Algorithm Analysis	Analyse the time and space requirements of the implemented Dijkstra solution.

1.6 Scope and Expected Outcome

The individual contribution covers the Dijkstra shortest-path and route-optimization layer of the complete navigation project. It includes initializing distance values, processing vertices, maintaining a priority queue, examining neighbouring edges and performing edge relaxation.

The implementation assumes that the graph contains non-negative edge weights, which is an important condition for Dijkstra's Algorithm. The algorithm calculates the minimum known distance from the selected source and updates distance values whenever a shorter path is discovered. The scope also includes verification of the calculated shortest-path distances using predefined test cases. The testing focuses on algorithm correctness, distance updates and the resulting minimum route cost.

The expected outcome is a functioning Dijkstra algorithm component that produces reliable shortest-path distances from a given source in a weighted navigation graph. The contribution is intended for an academic route-optimization system and does not include live traffic information, GPS positioning or dynamic road conditions.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The review considered common shortest-path and graph-processing approaches used for route optimization. The comparison focused on graph type, edge-weight requirements, shortest-path capability, computational characteristics and suitability for the navigation problem. The review particularly considered Dijkstra's Algorithm, Breadth-First Search, Bellman-Ford Algorithm and A* as alternative approaches to shortest-path computation.

2.2 Review of Existing Work

Different graph algorithms can be applied depending on the characteristics of the network and the required route-optimization problem. Breadth-First Search is appropriate for unweighted graphs, while weighted navigation networks require an algorithm that considers edge weights.

Dijkstra's Algorithm is suitable when edge weights are non-negative and the objective is to calculate minimum distances from a source. Bellman-Ford can handle negative edge weights but requires additional processing. A* can improve search efficiency when a suitable heuristic is available, but it introduces additional design requirements.

Different graph algorithms can be applied depending on the characteristics of the network and the requirements of the route-optimization problem. The selection of an appropriate shortest-path algorithm is important because different algorithms have different assumptions, computational requirements, and suitability for particular types of graphs. In a navigation system, roads can be represented as edges and locations as vertices, while the distance, travel cost, or travel time associated with each road can be represented as an edge weight. Therefore, the algorithm must be capable of considering these weights when determining the optimal route.

Existing Approach	Advantages	Limitations	Relevance to the System
Breadth-First Search	Simple and effective for unweighted graphs	Does not directly optimize weighted distances	Limited relevance
Bellman-Ford	Handles negative edge weights	More computationally expensive	Alternative for general weighted graphs
A* Algorithm	Can efficiently target a destination using a heuristic	Requires a suitable heuristic function	Possible future enhancement
Dijkstra's Algorithm	Suitable for non-negative weighted graphs; systematic shortest-path calculation	Does not handle negative edge weights	Selected for the project
Proposed Dijkstra system	Transparent calculations; priority-queue processing; easy verification	Designed for non-negative weights	Selected for academic navigation scope

2.3 Engineering Gap

The engineering gap addressed in this project is the need for a clear, efficient, and verifiable shortest-path implementation that can process a weighted navigation graph and determine the minimum route distance between a given source and destination. In a navigation network, multiple possible routes may exist between two locations, and each route may have a different total distance or cost. Therefore, simply identifying a connected path is not sufficient; the system must determine the path having the minimum accumulated weight.

Although several shortest-path algorithms are available, each algorithm is designed for particular graph conditions and has different computational characteristics. The proposed navigation problem primarily involves non-negative road weights, such as distance or travel cost. This creates a requirement for an algorithm that can efficiently process weighted edges while maintaining reliable shortest-distance calculations. Dijkstra's Algorithm is appropriate for this requirement because it is specifically designed to determine minimum distances from a source vertex when edge weights are non-negative. Another important gap is the need to demonstrate the internal working of the shortest-path calculation rather than treating route optimization as a black-box operation.

2.4 Proposed Contribution

My individual contribution addresses the Dijkstra Algorithm & Route Optimization layer of the system. The contribution focuses specifically on the design and implementation of Dijkstra's Algorithm for calculating minimum distances in the weighted navigation graph. The major responsibilities include distance initialization, priority-queue processing, selection of the minimum-distance vertex, edge relaxation, distance updating and shortest-path calculation.

The Dijkstra module receives the weighted graph and the selected source location as input. Each location in the navigation network is represented as a vertex, while the connecting roads are represented as weighted edges. The weights associated with the edges represent the distance or cost of travelling between connected locations. Since the navigation problem uses non-negative weights, Dijkstra's Algorithm can be applied to systematically determine the minimum accumulated distance from the source.

The implementation is developed using the C programming language, with the required graph-processing and priority-queue operations implemented using C data structures. Since C does not provide a built-in priority queue, a min-heap based priority queue is used to maintain vertices according to their tentative distance. This provides the required mechanism for efficiently selecting the vertex with the smallest known distance.

The algorithm begins by assigning a distance value of zero to the source vertex, because the distance from the source to itself is zero. All other vertices are assigned a large initial value representing an unknown or currently unreachable distance. This initialization allows the algorithm to distinguish between the source location and vertices whose minimum distances have not yet been determined.

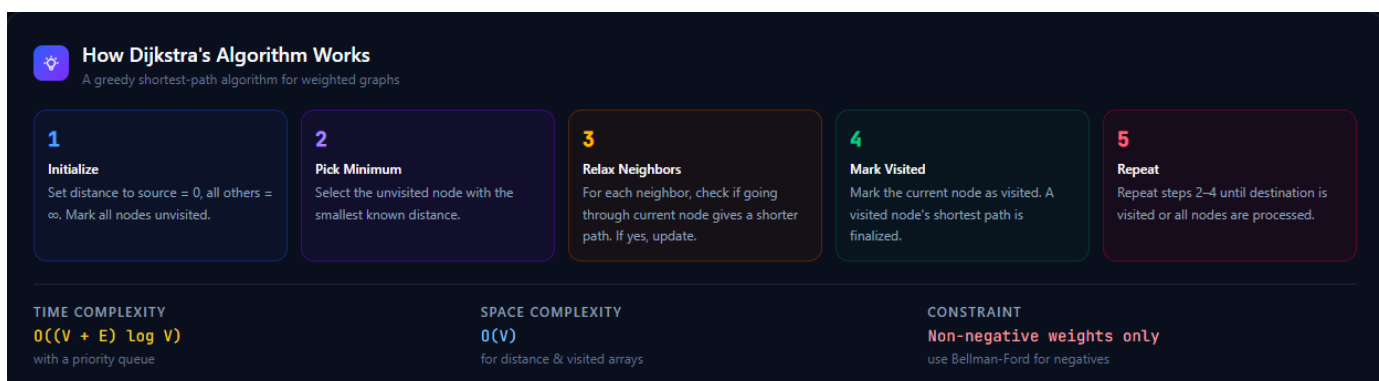


Figure 1 - Dijkstra algorithm and route-optimization module flow

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder	Requirement
Navigation user	Obtain an optimized route between selected locations.
System user	Receive the minimum route distance or cost.
Project evaluator	Verify correct implementation of Dijkstra's Algorithm.
Algorithm developer	Process weighted graph data using an appropriate shortest-path technique.
Requirement Type	Measurable Target
Source processing	Source vertex is initialized with distance 0.
Distance initialization	Other vertices begin with an appropriately large distance value.
Priority queue	Minimum tentative distance is selected for processing.
Edge relaxation	Shorter distances are identified and updated correctly.
Shortest-path calculation	Minimum route distance matches expected values.
Correctness	Test outputs match manually verified results.

3.2 Constraints & Applicable Practices

The Dijkstra component is constrained by the requirement to remain understandable, efficient and suitable for a Design and Analysis of Algorithms academic project.

The algorithm assumes that edge weights are non-negative. This constraint is important because Dijkstra's greedy selection relies on the fact that processing the currently closest vertex does not need to be reversed due to a negative-weight edge.

The implementation uses a priority queue to select the smallest tentative distance efficiently. The graph representation and supporting structures are treated as inputs to the Dijkstra module, allowing the individual contribution to focus specifically on shortest-path processing.

Correctness is maintained through explicit initialization, controlled distance updates and verification using known test cases.

3.3 Design Specifications

The design specification for my contribution is based on a shortest-path processing pipeline. The weighted graph is supplied to the Dijkstra module together with a source vertex. The algorithm initializes distances, inserts the source into the priority queue, selects the minimum-distance vertex and performs relaxation on its adjacent edges..

Parameter	Target / Requirement	Unit	Priority
Algorithm	Dijkstra's Algorithm	—	High
Edge weights	Non-negative	Distance / Cost	High
Source distance	0	Distance / Cost	High
Other initial distances	Infinity / large value	Distance / Cost	High
Processing structure	Priority queue / min-heap	—	High
Distance update	Edge relaxation	Distance / Cost	High
Output	Minimum distance	Distance / Cost	High

3.4 Alternative Solutions & Evaluation

Criterion	Dijkstra	BFS	Bellman-Ford	A*
Weighted graph support	High	Low	High	High
Non-negative weights	High	Not applicable	High	High
Negative weights	Low	Low	High	Low
Simplicity	High	High	Medium	Medium
Navigation relevance	High	Medium	Medium	High

3.5 Selected Design & Detailed Design

The selected design uses Dijkstra's Algorithm implemented in C with a min-heap based priority queue for shortest-path processing. The design was selected because the navigation problem consists of a weighted graph with non-negative edge weights and requires reliable minimum-distance calculation.

The Dijkstra module receives the weighted graph and the source vertex as input. An array is used to maintain the current shortest known distance for each vertex. Initially, the source vertex is assigned a distance of zero, while all other vertices are assigned a large value representing infinity or an unreachable state.

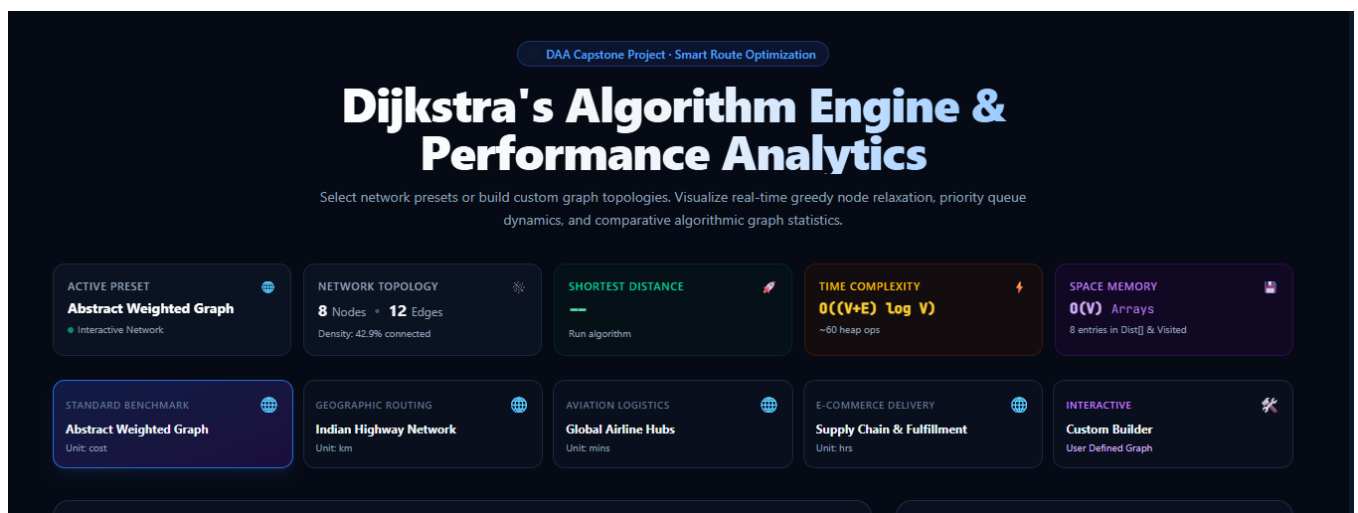


Figure 2. Dijkstra algorithm performance and complexity analysis

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Dijkstra's Algorithm	Bellman-Ford	Efficient for non-negative weighted graphs	Cannot handle negative edges	Selected because navigation weights are non-negative
Priority queue	Linear minimum search	Faster minimum-element selection	Additional data structure	Selected for efficient processing
Distance array/vector	Complex database structure	Simple and transparent	Limited to current graph representation	Selected for academic implementation
Edge relaxation	Manual route comparison	Systematic shortest-path calculation	Requires careful implementation	Selected because it is the core Dijkstra operation

3.7 Sustainability, Ethics, Safety, Risk & Security

The Dijkstra component is lightweight and requires limited computational infrastructure. Using standard algorithmic data structures reduces unnecessary resource requirements and makes the system suitable for academic experimentation.

Ethical considerations include ensuring that route outputs are not presented as guaranteed real-world navigation decisions when the system does not include live traffic, road closures or dynamic conditions.

From a safety perspective, incorrect shortest-path calculations could result in an unsuitable route if the system were used in a real navigation environment. Therefore, algorithm outputs must be verified through test cases before interpretation.

Security considerations include protecting any navigation-related input data and preventing unauthorized modification of the graph or route information.

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Use lightweight algorithmic processing	Standard C++ data structures	Reduced infrastructure requirements
Ethics	Avoid misleading route claims	Testing and defined project scope	Results interpreted within system limitations
Safety	Avoid incorrect shortest-path results	Manual and test-case verification	Testing required before use
Security	Protect graph and route data	Controlled project environment	Suitable for academic prototype

Risk Register

Important risks include incorrect distance initialization, failure to perform edge relaxation correctly, incorrect priority-queue processing, invalid graph weights and incorrect shortest-path outputs. Mitigation includes input validation, controlled initialization, algorithm tracing and manual verification of test results.

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Incorrect distance initialization	Medium	High	High	Verify source and initial values	Low
Incorrect edge relaxation	Medium	High	High	Trace relaxation calculations	Low
Priority queue processing error	Medium	Medium	Medium	Test minimum-distance selection	Low
Negative edge weight	Low	High	Medium	Validate edge-weight constraint	Low
Incorrect shortest-path result	Medium	High	High	Compare with manually calculated result	Low
Invalid source/destination	Low	Medium	Low	Input validation	Low

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The development of the Dijkstra Algorithm & Route Optimization module was carried out using a structured algorithm-development approach. The implementation was divided into individual stages so that each operation of the shortest-path algorithm could be developed and verified independently. The primary focus was on implementing Dijkstra's Algorithm in the C programming language for a weighted navigation graph containing non-negative edge weights.

The development process began with identifying the inputs and outputs required by the Dijkstra module. The input consists of the weighted navigation graph and a selected source vertex. The output consists of the minimum distance from the source to each reachable vertex and, where required, the minimum route distance to the selected destination.

Resources Used

Resource	Purpose
C Programming Language	Implementation of Dijkstra's Algorithm
C Compiler	Compilation and execution of the program
Structures	Representation of graph and priority-queue elements
Arrays	Storage of vertices, distances and supporting information
Min-Heap	Implementation of the priority queue
Test Graphs	Verification of shortest-path calculations
Microsoft Word	Technical report preparation

4.2 Software Implementation & Modern Tools Used

The Dijkstra Algorithm and route-optimization module was implemented entirely using the C programming language. C was selected for the implementation because it provides direct control over arrays, structures, memory and data structures required for implementing graph algorithms.

The implementation does not depend on a built-in priority-queue library. Instead, the required priority queue is constructed using a min-heap. This provides direct implementation experience with the data structure and demonstrates how a priority queue can be integrated with Dijkstra's Algorithm.

Tool / Technology	Purpose
C	Core algorithm implementation
C Compiler	Compiling and executing the program
Structures in C	Representing graph and heap elements
Arrays in C	Maintaining distances and graph-related information
Min-Heap	Priority queue for selecting minimum-distance vertices
IDE / Code Editor	Writing, modifying and debugging the C program
Microsoft Word	Documentation and report preparation

4.3 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion
Distance initialization	Set source distance to 0 and remaining vertices to a large value	Source is initialized correctly
Minimum vertex selection	Process vertices using the min-heap	Vertex with smallest tentative distance is selected
Edge relaxation	Calculate new distances through connected edges	Shorter distance is identified and updated
Distance update	Compare calculated distance with stored distance	Stored value is replaced only when a shorter route is found
Shortest distance	Compare program output with manually calculated route distance	Calculated distance matches expected minimum distance
Multiple routes	Provide more than one route between source and destination	Minimum weighted route is selected

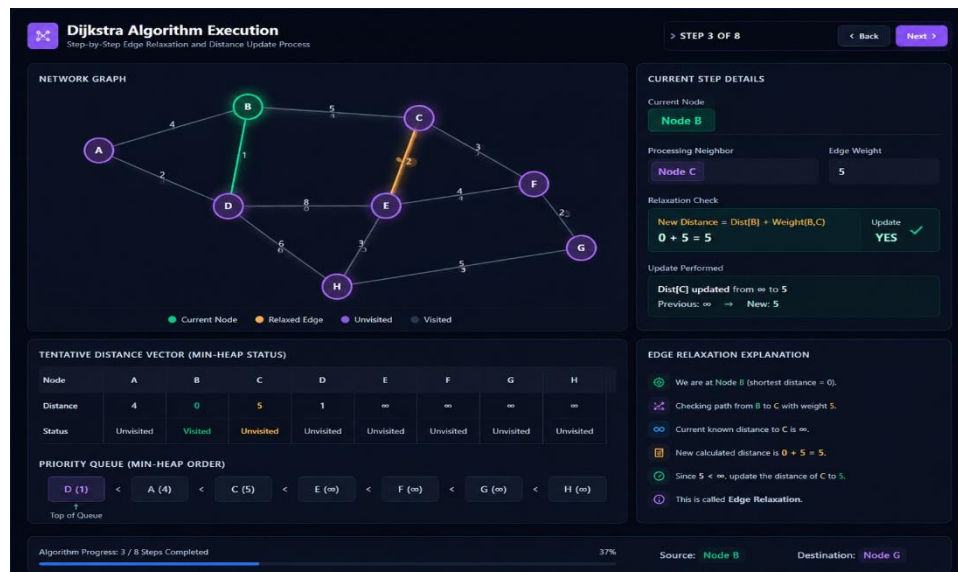


Figure 3. Edge Relaxation and Distance Update Process

4.4 Results

The implemented Dijkstra Algorithm & Route Optimization module calculates the shortest route between a selected source node and destination node in the weighted navigation graph. The execution interface provides a visual representation of the graph, selected nodes, route weights and the resulting shortest path.

For the tested case, Node B is selected as the source and Node G as the destination. The algorithm determines the shortest path as Node B → Node C → Node F → Node G, with a total route cost of 10 units. The execution results also indicate that 8 out of 8 nodes were visited during the processing.

The displayed visit order provides evidence of the sequence in which the vertices were processed by the algorithm. The tentative distance vector records the minimum known distance for each node as the algorithm progresses. These values demonstrate the effect of distance initialization, minimum-distance selection and edge relaxation during Dijkstra's execution.

The results confirm that the module can process a weighted graph and identify the minimum-cost route between the specified source and destination. The graphical visualization also makes the selected shortest path and its corresponding route cost directly verifiable.

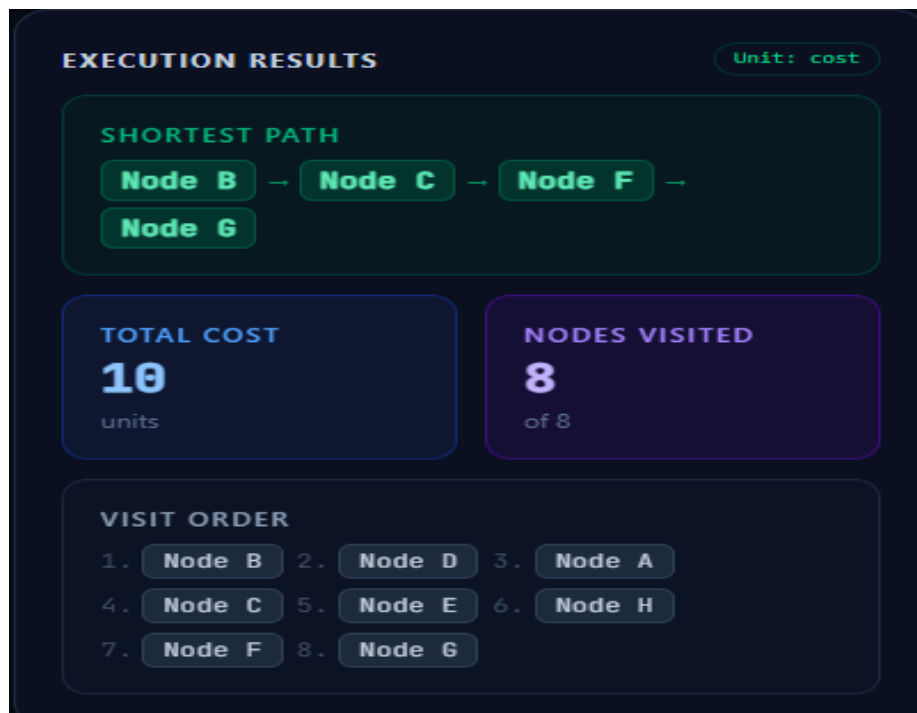


Figure 4 - Shortest-path calculation output

4.5 Data Analysis & Engineering Judgment

The main engineering judgment in this contribution was to select Dijkstra's Algorithm for the weighted navigation graph containing non-negative edge weights. This selection is appropriate for the route-optimization problem because road distances and route costs are represented using non-negative values, allowing the algorithm to determine minimum-cost routes from a selected source to the required destination.

The use of a min-heap based priority queue provides an efficient mechanism for selecting the vertex with the smallest tentative distance during each stage of processing. This avoids unnecessary selection operations and supports efficient progression through the weighted graph.

The distance array maintains the shortest known distance for each vertex at every stage of execution. Initially, the source vertex is assigned a distance of zero, while the remaining vertices are assigned large initial values. As the algorithm processes the graph, these values are updated whenever a shorter route is identified.

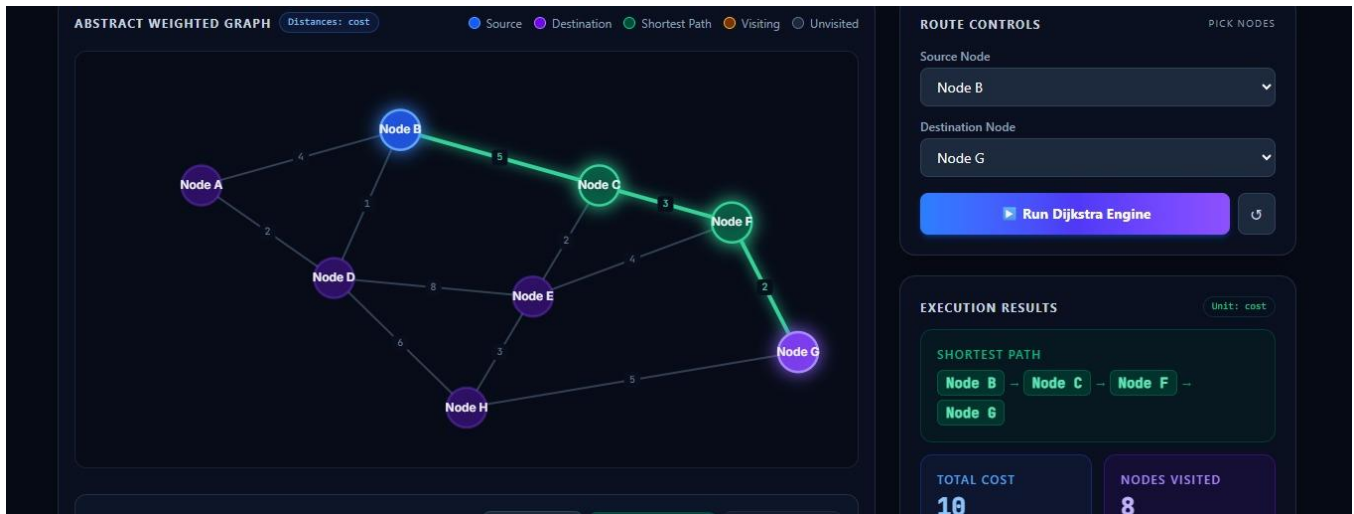


Figure 5. Route Optimization Result

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Calculate minimum route distance	Dijkstra shortest-path processing	Fully Achieved
Process weighted navigation graph	Weighted vertices and edges are processed	Fully Achieved
Select minimum-distance vertex	Min-heap based priority queue	Fully Achieved
Perform edge relaxation	Neighbour distances are compared and updated	Fully Achieved

4.7 Strengths & Limitations

Strengths: The Dijkstra component provides a clear and systematic method for calculating minimum route distances in a weighted navigation graph. The use of a min-heap supports efficient minimum-distance vertex selection, while edge relaxation allows shorter routes to be identified and stored. The processing sequence is also transparent and can be verified using individual graph edges and distance updates.

Limitations: Dijkstra's Algorithm is applicable only when the graph contains non-negative edge weights. The current implementation also works with the graph and weights provided to the system and does not independently account for continuously changing real-world conditions such as live traffic, road closures or changing travel times.

4.8 Project Planning, Roles & Budget

The project was developed in separate modules so that each team member could focus on an individual technical contribution. My individual contribution was Part 2 – Dijkstra Algorithm & Route Optimization.

The work included the implementation of distance initialization, min-heap based priority-queue processing, minimum-distance vertex selection, edge relaxation, distance updates and shortest-distance calculation.

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

The Dijkstra Algorithm & Route Optimization module successfully implements shortest-path processing for a weighted navigation graph with non-negative edge weights. The module uses distance initialization, minimum-distance vertex selection, min-heap based priority-queue processing, edge relaxation and distance updates to determine minimum route distances.

The implementation demonstrates the practical application of important Design and Analysis of Algorithms (DAA) concepts, particularly the greedy approach and shortest-path computation. The results obtained from testing confirm that the algorithm can identify minimum route distances among multiple possible paths.

5.2 Future Work

The Dijkstra-based route-optimization module can be extended to support more advanced navigation requirements. The current implementation can be further developed to incorporate dynamic road conditions, changing travel costs and real-time traffic information, allowing route calculations to respond to changes in the navigation environment.

The route-optimization process could also be extended to consider additional factors such as travel time, road conditions and changing route weights instead of relying only on fixed edge costs. This would allow the system to provide more practical route recommendations under varying traffic and road conditions.

For larger and more complex navigation networks, the module could be integrated with advanced navigation techniques to improve route-selection efficiency. Such extensions can help the system handle changing network conditions and support more efficient shortest-path computation in real-world navigation scenarios.

5.3 Professional Learning & Individual Reflection

Working on the Dijkstra Algorithm & Route Optimization module provided practical experience in applying shortest-path algorithms to a navigation problem. The implementation helped in understanding how a weighted graph can be processed to determine minimum route distances.

The major learning outcomes included understanding greedy selection, distance initialization, priority-queue processing, min-heap operations, edge relaxation and distance updates. Implementing these operations in C also provided practical experience in developing and testing algorithmic data structures.

The testing process helped in understanding the importance of verifying each distance update and comparing the final results with expected shortest-path values.

Overall, this contribution improved my understanding of how theoretical DAA concepts can be translated into a practical route-optimization solution using the C programming language.

References

- [1] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, “Introduction to Algorithms,” MIT Press.
- [2] Robert Sedgewick and Kevin Wayne, “Algorithms,” Addison-Wesley.
- [3] C Programming Language documentation and standard library reference.
- [4] ISO/IEC 9899, “Programming Languages — C.”
- [5] Project implementation and shortest-path test data, Smart Route Optimization Using Dijkstra’s Algorithm for Intelligent Navigation Systems, 2026.

APPENDICES

Appendix A – Project Architecture / Evidence

```
#include <stdio.h>

#include <stdlib.h>

#include <limits.h>
```

```
#define MAX 100
```

```
typedef struct {
    int vertex;
    int distance;
} HeapNode;
```

```
typedef struct {
    HeapNode heap[MAX];
    int size;
} MinHeap;
```

```
typedef struct {
    int vertex;
    int weight;
    int next;
} Edge;
```

```

Edge edges[MAX * MAX];

int head[MAX];

int edgeCount = 0;

/* Add an edge to the graph */
void addEdge(int u, int v, int weight) {
    edges[edgeCount].vertex = v;
    edges[edgeCount].weight = weight;
    edges[edgeCount].next = head[u];
    head[u] = edgeCount++;
}

/* Swap two heap nodes */
void swap(HeapNode *a, HeapNode *b) {
    HeapNode temp = *a;
    *a = *b;
    *b = temp;
}

/* Insert an element into the min-heap */
void insertHeap(MinHeap *pq, int vertex, int distance) {
    int i = pq->size++;

    pq->heap[i].vertex = vertex;
    pq->heap[i].distance = distance;

```

```

while (i > 0) {
    int parent = (i - 1) / 2;

    if (pq->heap[parent].distance <= pq->heap[i].distance)
        break;

    swap(&pq->heap[parent], &pq->heap[i]);
    i = parent;
}
}

```

/* Remove the minimum-distance element */

```

HeapNode extractMin(MinHeap *pq) {
    HeapNode result = pq->heap[0];

    pq->size--;

    if (pq->size > 0) {
        pq->heap[0] = pq->heap[pq->size];

        int i = 0;

        while (1) {
            int left = 2 * i + 1;
            int right = 2 * i + 2;
            int smallest = i;

```

```

    if (left < pq->size &&
        pq->heap[left].distance < pq->heap[smallest].distance)
        smallest = left;

    if (right < pq->size &&
        pq->heap[right].distance < pq->heap[smallest].distance)
        smallest = right;

    if (smallest == i)
        break;

    swap(&pq->heap[i], &pq->heap[smallest]);
    i = smallest;
}

return result;
}

/* Dijkstra's shortest-path algorithm */
void dijkstra(int vertices, int source) {
    int distance[MAX];

    MinHeap pq;
    pq.size = 0;

    /* Distance initialization */

```

```

for (int i = 0; i < vertices; i++)
    distance[i] = INT_MAX;

distance[source] = 0;

/* Insert source into priority queue */
insertHeap(&pq, source, 0);

while (pq.size > 0) {
    HeapNode current = extractMin(&pq);

    int u = current.vertex;

    /* Ignore outdated heap entries */
    if (current.distance != distance[u])
        continue;

    /* Edge relaxation */
    for (int i = head[u]; i != -1; i = edges[i].next) {
        int v = edges[i].vertex;
        int weight = edges[i].weight;

        if (distance[u] != INT_MAX &&
            distance[u] + weight < distance[v]) {

            distance[v] = distance[u] + weight;

```



```

        insertHeap(&pq, v, distance[v]);
    }
}

/* Display shortest distances */

printf("\nShortest distances from source %d:\n", source);

for (int i = 0; i < vertices; i++) {
    if (distance[i] == INT_MAX)
        printf("Vertex %d : Unreachable\n", i);
    else
        printf("Vertex %d : %d\n", i, distance[i]);
}

/* Main function */

int main() {
    int vertices, edgesCount;
    int source;

    printf("Enter number of vertices: ");
    scanf("%d", &vertices);

    for (int i = 0; i < vertices; i++)
        head[i] = -1;

```

```
printf("Enter number of edges: ");  
scanf("%d", &edgesCount);  
  
printf("Enter edges as: source destination weight\n");  
  
for (int i = 0; i < edgesCount; i++) {  
    int u, v, weight;  
  
    scanf("%d %d %d", &u, &v, &weight);  
  
    addEdge(u, v, weight);  
}  
  
printf("Enter source vertex: ");  
scanf("%d", &source);  
  
dijkstra(vertices, source);  
  
return 0;  
}
```

Appendix B – TEST CASES

Dijkstra Algorithm & Route Optimization

Test Case	Input Condition	Expected Output	Result
TC01	Source vertex selected	Source distance = 0	Pass
TC02	All vertices initialized	Non-source vertices set to INF	Pass
TC03	Direct route exists	Direct edge distance calculated correctly	Pass
TC04	Multiple routes exist	Minimum weighted route selected	Pass
TC05	Shorter route discovered	Existing distance updated	Pass
TC06	Multiple vertices in priority queue	Minimum-distance vertex selected first	Pass
TC07	Unreachable vertex	Vertex reported as unreachable	Pass
TC08	Weighted graph with non-negative edges	Correct shortest distances calculated	Pass

Test Result

The test cases verify the major operations of the Dijkstra module, including distance initialization, minimum-distance vertex selection, edge relaxation, distance updating and shortest-distance calculation. The results confirm that the implemented algorithm processes the weighted navigation graph according to the expected Dijkstra procedure.

CAPSTONE PROJECT OUTCOME MAPPING SHEET

Outcome	Evidence / SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.7)
Project planning and management	SO5	Ch. 3 (3.5), Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)